ALGORITMOS Y PROGRAMACIÓN II

Clase 5
Recursión

Temas de la clase

1. Recursión

2. Módulo recursivo

3. Pila de ejecución

Recursión

- Es una técnica de programación que permite resolver problemas en términos de si mismo:

MiProblema(estado_N) = solución_de_una_parte + MiProblema(estado_N-1)

- La recursión es una técnica de programación muy poderosa, en la cual un módulo realiza llamadas a si mismo en pos de resolver un problema.
- Razones para su uso:
  - Problemas "casi" irresolubles con las estructuras iterativas.
  - Soluciones elegantes y simples

Recursión

MiProblema(estado_N) = solución_de_una_parte + MiProblema(estado_N-1)

- Los módulos recursivos bien definidos poseen dos elementos:
  - **Caso Recursivo**: aquí el módulo realiza algunas operaciones con las que se reduce la complejidad del problema y luego realiza un llamado a si mismo.
  - **Caso Base**: Se da cuando el cálculo es tan simple que se puede resolver directamente sin necesidad de hacer una llamada recursiva (fin parcial o total de la solución).

Recursión

Ejemplo: Cálculo de la sumatoria de N:

$$\text{Sumatoria(N)} = \sum_{i=1}^{N} i = 1 + 2 + 3 + \cdots + (i-2) + (i-1) + i$$

$$\text{Sumatoria}(5) = 1 + 2 + 3 + 4 + 5 = 15$$

Solución NO-recursiva

```
function sumatoria(N: integer): integer;
var sum, i: integer;
begin
sum:= 0;
for i:= 1 to N do
  sum:= sum + i;
```

¿Cómo es la solución recursiva al problema de la sumatoria de N?

Recursión

Ejemplo: Cálculo recursivo de la sumatoria de 5.

Sumatoria(5) = 1 +2 + 3 + 4 + 5 = 15

Sumatoria(4) = 1 + 2 + 3 + 4 = 10

Sumatoria(3) = 1 + 2 + 3 = 6

Sumatoria(2) = 1 +2 = 3

Sumatoria(1) = 1

Caso base

Recursión

En términos generales:

Sumatoria(N) = Sumatoria(N-1) + N

Sumatoria(1) = 1

Solución recursiva:

```
function sumatoria_rec(N: integer): integer;
begin
if N <> 1 then
    sumatoria_rec:= sumatoria_rec(N-1) + N
else
    sumatoria_rec:= 1;
end;
```

Caso recursivo
(la función se invoca a si misma)

Caso base
(fin de la recursión)

Actividades en Máquina

ACTIVIDAD 1

1. Implemente el módulo **sumatoria_rec**.
2. Haga un programa que lea un número natural mayor a 0 por teclado y que imprima el resultado de la sumatoria del número leído.

¿Qué le sucede al programa si se ingresa el valor cero o un número negativo?

¿Qué le sucede al programa si se ingresa un número mayor a 445?

Recursión sobre estructuras de datos

Función que devuelve la suma de todos los elementos de un vector de reales:

```
const DIMF = 100;
type
   vector = array[1..DIMF] of real;
{ Solución NO recursiva }
   function sumaTotal(v: vector): real;
   var sum: real;
      i: integer;
   begin
   sum:=0;
   for i:= 1 to DIMF do
      sum:= sum + v[i];
   sumaTotal:= sum;
   end;
```

¿Cómo se implementa la versión recursiva?

Recursión sobre estructuras de datos

Ejemplo: Suma de todos los valores de un vector:

V = | 3 | 4 | 1 | 5 | 2 |

sumaTotal(V) = V[1]+V[2]+V[3]+V[4]+V[5]

sumaTotal([3, 4, 1, 5]) = V[1] + V[2] + V[3] + V[4]

sumaTotal([3, 4, 1]) = V[1] + V[2] + V[3]

sumaTotal([3, 4]) = V[1] + V[2]

sumaTotal([3]) = V[1]

Caso base

Recursión sobre estructuras de datos

Ejemplo: Suma de todos los valores de un vector:

V = | 3 | 4 | 1 | 5 | 2 |

Solución recursiva

```
function sumaTotal_rec(v: vector; i: integer): real;
begin
if i > 1 then
   sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
else
   sumaTotal_rec:= v[i];
end;
```

Actividades en Máquina

ACTIVIDAD 2

1. Implemente el módulo **sumaTotal_rec**.
2. Haga un programa que cargue un vector de 100 reales y que imprima el resultado de la sumatoria de todos los valores almacenados en el vector.

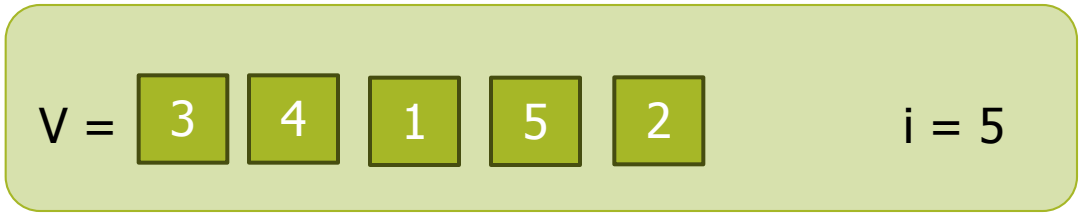
¿Qué le sucede al programa si se pide calcular la suma de todos los valores de un vector de 20000 elementos?

Pila de ejecución

Ejemplo: Suma de todos los valores de un vector:

```
{Programa principal}
var v: vector;
Begin
cargarVector(v);
writeln(sumaTotal_rec(v, DIMF));
end;
```

Al invocar a un módulo (recursivo o no) se reserva memoria para todas las variables locales del módulo y sus parámetros.

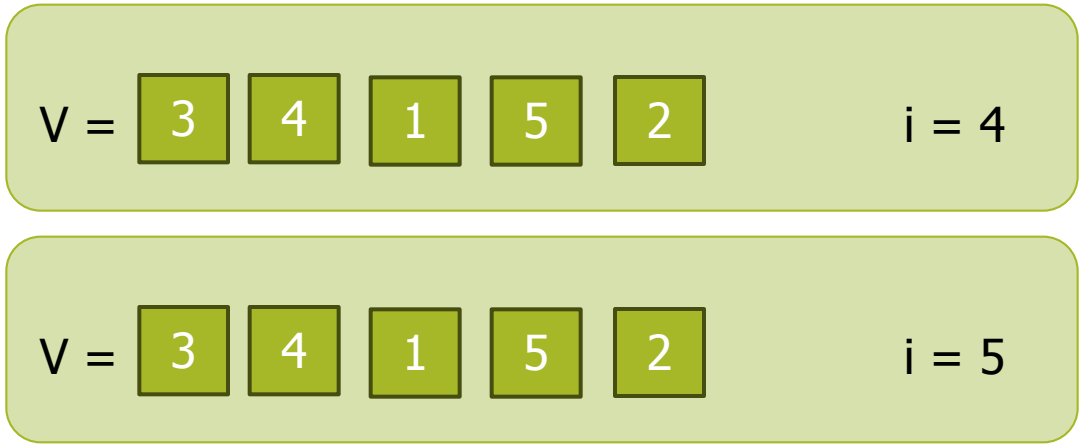


Memoria

Pila de ejecución

```
function sumaTotal_rec(v: vector; i: integer): real;
begin
if i > 1 then
    sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
else
    sumaTotal_rec    v[i];
end;
```

Al hacer la llamada recursiva, vuelve a pasar lo mismo: se reserva memoria para todas las variables locales del módulo y sus parámetros. **¡¡¡¡¡Pero la anterior llamada aún no finalizó!!!!!**



Memoria

Pila de ejecución

Con cada llamada recursiva sucede lo mismo.
Y esto sucede hasta alcanzar el caso base.

Lo que sucede en memoria se conoce como pila de ejecución.

Notar que hasta este punto aún no se ha realizado ninguna suma.

V = 3 4 1 5 2 i = 1

V = 3 4 1 5 2 i = 2

V = 3 4 1 5 2 i = 3

V = 3 4 1 5 2 i = 4

V = 3 4 1 5 2 i = 5

Memoria

Pila de ejecución (deshaciendo el camino)

Se alcanzó el caso base. Se devuelve v[i] = 3

```
...
else
sumaTotal_rec:= v[i];
end;
```

V = 3 4 1 5 2 i = 1

V = 3 4 1 5 2 i = 2

V = 3 4 1 5 2 i = 3

V = 3 4 1 5 2 i = 4

V = 3 4 1 5 2 i = 5

Memoria

Pila de ejecución (deshaciendo el camino)

```
if i > 1 then
    sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
```

Recibe el 3 de la llamada anterior y le suma v[2]
3 + 4 = 7

V = 3 4 1 5 2 i = 2

V = 3 4 1 5 2 i = 3

V = 3 4 1 5 2 i = 4

V = 3 4 1 5 2 i = 5

Memoria

Pila de ejecución (deshaciendo el camino)

```
if i > 1 then
    sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
```

Recibe el 7 de la llamada anterior y le suma v[3]
7 + 1 = 8

V = 3 4 1 5 2 i = 3

V = 3 4 1 5 2 i = 4

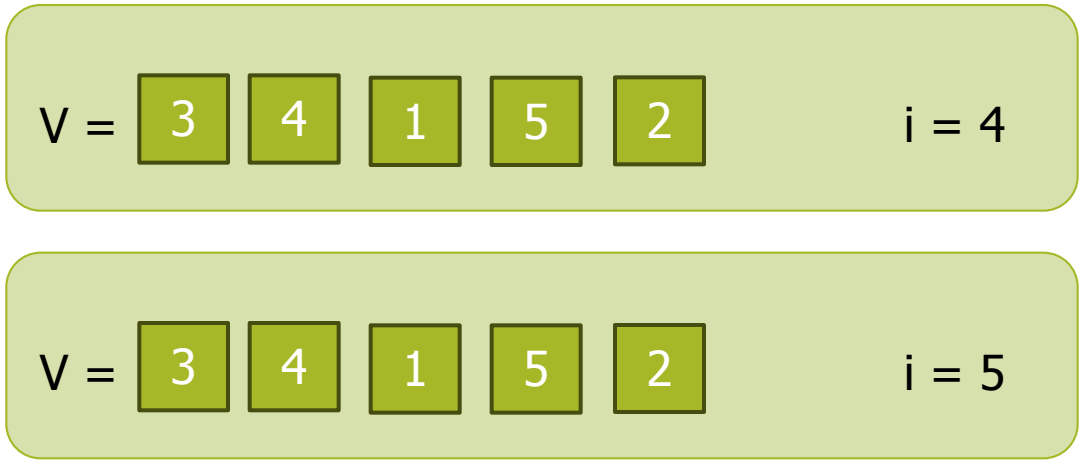
V = 3 4 1 5 2 i = 5

Memoria

Pila de ejecución (deshaciendo el camino)

```
if i > 1 then
    sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
```

Recibe el 8 de la llamada anterior y le suma v[4]
8 + 5 = 13



Memoria

Pila de ejecución (deshaciendo el camino)

```
if i > 1 then
    sumaTotal_rec:= sumaTotal_rec(v,i-1) + v[i]
```

¿Quién recibe el 15?

Recibe el 13 de la llamada anterior y le suma v[5]
13 + 2 = 15

V = | 3 | 4 | 1 | 5 | 2 | i = 5

Memoria

Pila de ejecución (deshaciendo el camino)

El gran problema de las soluciones recursivas es el excesivo uso de memoria que necesitan para ejecutarse.

```
{Programa principal}
var v: vector;
Begin
cargarVector(v);
writeln(sumaTotal_rec(v, DIMF));
end;
```

La llamada original recibe el 15 y lo imprime por pantalla.

¿Podemos resolver este problema ahorrando algo de memoria para poder hacer más llamadas recursivas?

Actividades en Máquina

ACTIVIDAD 3

1. Implemente un módulo **recursivo** que reciba un vector de reales e imprima todos sus valores en el mismo orden en que están cargados.
2. Implemente un módulo **recursivo** que reciba un vector de reales e imprima todos los valores en orden inverso en el que están cargados.
3. Haga un programa que cargue un vector de 10 reales y que imprima sus elementos en el orden en que fueron cargados, y en el orden inverso.

Recursión sobre estructuras de datos

Función que devuelve la suma de todos los elementos de una lista de reales:

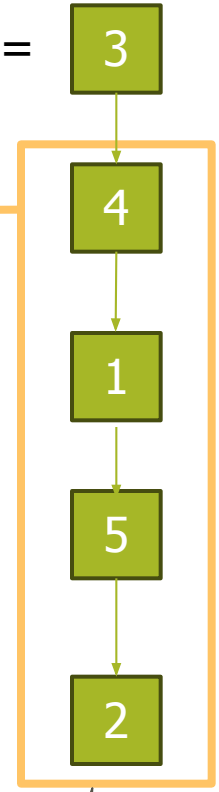
```
type
   lista = specialize LinkedList<real>;
{ Solución NO recursiva }
   function sumaTotal(l: lista): real;
   var sum: real;
   begin
   l.reset();
   while not l.eol() do begin
      sum:= sum + l.current();
      l.next();
      end;
   sumaTotal:= sum;
   end;
```

¿Cómo se implementa la versión recursiva?

Recursión sobre estructuras de datos

Ejemplo: Suma de todos los valores de una lista:

L =



sumaTotal(L) = 3 + sumaTotal([4, 1, 5, 2])

4 + sumaTotal([1, 5, 2])

1 + sumaTotal([5, 2])

5 + sumaTotal([2])

2 + sumaTotal([])

Caso base
(lista vacía)

Sublista

Recursión sobre estructuras de datos

Ejemplo: Suma de todos los valores de una lista:

L =

Solución recursiva

```
function sumaTotal_rec(l: lista): real;
var res: real;
begin
if not l.eol() then  begin
   res:= l.current();
   sumaTotal_rec:= res + sumaTotal_rec(l.getSubList());
   end
else
   sumaTotal_rec:= 0;
end;
```

Devuelve la sublista (Desde el segundo nodo hasta el final)

Actividades en Máquina

ACTIVIDAD 4

1. Implemente el módulo **sumaTotal_rec**.
2. Haga un programa que cargue una lista de 100 números reales y que imprima el resultado de la sumatoria de todos los valores almacenados en la lista.

Actividades en Máquina

ACTIVIDAD 5

1. Implemente un módulo **recursivo** que reciba una lista de reales e imprima todos sus valores en el mismo orden en que están cargados.
2. Implemente un módulo **recursivo** que reciba una lista de reales e imprima todos los valores en orden inverso en el que están cargados.
3. Haga un programa que cargue una lista de 10 reales y que imprima sus elementos en el orden en que fueron cargados, y en el orden inverso.

Actividades en Máquina

ACTIVIDAD 6

1. Implemente una versión recursiva del módulo **buscarMinimo** de la operación merge.
2. Modifique el programa de la actividad 2 de la clase 3 para comprobar el correcto funcionamiento del módulo implementado en esta actividad.